

An Enhanced Data Packing Method for General Matrix Multiplication in Brakerski/Fan-Vercauteren Scheme

Xiangchen Meng, Yan Tan, Zijun Jiang and Yangdi Lyu*

Microelectronics Thrust, The Hong Kong University of Science and Technology (Guangzhou)
Guangzhou, Guangdong, China

Emails: {xmeng027, ytan910, zjiang438}@connect.hkust-gz.edu.cn, yangdilyu@hkust-gz.edu.cn

Abstract—General Matrix-Matrix Multiplication (GEMM) stands as the most ubiquitous operation in machine learning applications. However, performing GEMM within Fully Homomorphic Encryption (FHE) is inefficient due to high computational demands and significant data migration constrained by limited bandwidth. Additionally, the inherent limitations of FHE schemes restrict the widespread application of machine learning, as standard activation functions are incompatible. This incompatibility necessitates alternative nonlinear functions, which lead to notable accuracy reductions. To address these challenges, we introduce a polynomial encoding methodology for GEMM under the Brakerski/Fan-Vercauteren (BFV) scheme and extend the method to inference with packing inputs and weights for different sizes. Furthermore, we design specialized hardware to accelerate the inference process through optimized scheduling between the hardware and the host system. In experiments, we implemented our hardware on an FPGA U250 platform. Compared to existing solutions, our method achieves superior performance, achieving the highest $4.22\times$ and $3.99\times$ speedups on MNIST and CIFAR-10.

Index Terms—Fully Homomorphic Encryption, FPGA, Matrix Multiplication

I. INTRODUCTION

The intensive tasks based on cloud platforms offer unparalleled scalability and computational prowess, rendering them ideal for training complex models and processing vast datasets. Yet in sensitive domains like healthcare, finance, and personalized services, data privacy concerns significantly impede the adoption of cloud-based solutions. To address these challenges, various privacy-preserving methodologies have been proposed for machine learning tasks, including differential privacy [1], secure multi-party computation [2], federated learning [3], and trusted execution environments [4]. Among these techniques, FHE has garnered significant attention from both academia and industry due to its unique capability to perform computations directly on encrypted data without the need for decryption. This property makes FHE particularly suitable for privacy-preserving machine learning applications, as it allows for secure model inference on sensitive data.

However, current implementations of CNN models under FHE face significant bottlenecks that hinder practical adoption:

Accuracy Degradation: Inference based on FHE limited by fundamental constraints, which couldn't use non-linear activation functions, such as *ReLU*, *Sigmoid* et al. To maintain homomorphic encryption compatibility, existing solutions employ the *square* as alternative activation functions that introduce a degradation in model accuracy [5].

Homomorphic Latency: Homomorphic computation involves numerous complex polynomial calculations, resulting in operations that run 4 to 5 orders of magnitude slower than unencrypted counterparts. This substantial performance gap severely limits the feasibility of real-time machine learning applications [6], [7].

Data Proliferation: The encryption process transforms data into high-degree polynomials (ranging from 2^{10} to 2^{16}) with each coefficient reaching hundreds of bits [8], [9]. This transformation causes substantial data expansion, leading to significant storage and bandwidth overhead.

To address these challenges, extensive research efforts have been devoted to mitigating the computational bottlenecks of FHE in machine learning applications. Initial approaches focused on CPU-based implementations with limited parallelism capabilities in handling the complexity of FHE operations. Seminal work by CryptoNets [10] established the foundational framework for privacy-preserving inference using homomorphic encryption, but the substantial computational overhead, requiring approximately 205 seconds for inference completion. Subsequently, Lola [5] introduced an innovative approach that categorized data types based on layer-wise intermediate results, implementing specialized operations for each category. This optimization yielded significant performance improvements, reducing inference latency to 2.2 seconds on the MNIST dataset relative to CryptoNets. However, the speed is still not fast enough to meet the requirements of real-time applications. Meanwhile, GPU-based implementations have been explored at the same time. Although optimized for tensor operations, which could achieve higher parallel, it is not optimized for the modular arithmetic intrinsic to homomorphic encryption. For instance, TensorFHE [11], despite leveraging GPU acceleration for homomorphic operations in convolutional neural networks, achieves a large improvement over CPU. But homomorphic multiplication operation (*HMULT*) still exhibits a substantial latency of 851 milliseconds.

To overcome the inherent hardware limitations, the research community has increasingly explored FPGA-based architectures for accelerating homomorphic encryption computations [12]–[14]. FPGAs offer distinct advantages through their reconfigurable hardware architecture, facilitating customized optimizations for modular arithmetic operations while maintaining high parallelism and computational efficiency. Cheetah [15], which introduced specialized optimizations for FPGA-based inference. Through weight duplication, Cheetah minimized rotation operations in dot-product computations, thereby enhancing computational efficiency and reducing noise accumulation during processing. However, this approach encounters scalability challenges with larger networks due to increased memory bandwidth requirements and data transfer overhead. FxHENN [16] advanced the field by implementing detailed hardware optimizations for inference. It utilized design space exploration layer by layer to maximize memory usage and computational efficiency. However, FxHENN processes each pixel individually during convolution operations, which increases memory consumption and computational complexity because it fails to leverage data reuse and parallel processing opportunities, resulting in higher system overhead.

*Corresponding Author: yangdilyu@hkust-gz.edu.cn

Therefore, we propose an enhanced data-packing method, to speed up matrix multiplication and relieve bandwidth pressure. Our key contributions are as follows:

- 1) We introduce an enhanced data-packaging methodology for general matrix multiplication under FHE, reducing both memory footprint and computational complexity.
- 2) We extend the method to convolutional layers and fully connected layers and dynamically plan the packing method based on the model parameters.
- 3) We design hardware based on FPGA and plan the pipeline to set up scheduling between the host and FPGA
- 4) The experiment on Xilinx FPGA platforms for two CNN models demonstrates the significant outperforms compared with existing solutions, achieving $4.22\times$ and $3.99\times$ speed up with higher accuracy.

II. BACKGROUND

A. Fully Homomorphic Encryption

FHE is a cryptographic technique that allows computations to be performed directly on encrypted data without requiring decryption. One widely used FHE scheme is the BFV scheme [17], [18]. In the BFV scheme, a plaintext message m is represented as a polynomial in the ring $R_t = \mathbb{Z}_t[x]/(x^n+1)$, where n denotes the polynomial degree and t is a plaintext modulus. This polynomial m can be expressed as:

$$m(x) = m_0 + m_1x + m_2x^2 + \dots + m_{n-1}x^{n-1}$$

where $m_i \in \mathbb{Z}_t$ are the coefficients of the plaintext polynomial. The commonly used data encoding method of BFV is batch encoding, which allows multiple plaintext messages to be encoded into a single polynomial, thereby enabling parallel homomorphic computations on these elements.

The encryption process involves several key steps. First, the plaintext polynomial m is scaled by a large integer Δ to ensure noise tolerance. Then, two random polynomials a and e are sampled from a discrete Gaussian distribution. This ciphertext $ct = (c_0, c_1)$ is generated as :

$$c_0 = -a \cdot s + e_1 + \Delta \cdot m \pmod{q}$$

$$c_1 = -a$$

where s is the secret key polynomial, a and e are randomly chosen polynomials from the ring $R_q = \mathbb{Z}_q[x]/(x^n+1)$. The parameter q is a large modulus that determines the noise capacity of the scheme. The BFV scheme supports two primary homomorphic operations enabling computations on encrypted data. These operations are defined as follows:

Homomorphic Addition: Given two ciphertexts ct_1 and ct_2 corresponding to plaintexts m_1 and m_2 , homomorphic addition is defined as:

$$Dec(ct_1 + ct_2) = m_1 + m_2$$

Homomorphic Multiplication: Similarly, for homomorphic multiplication of ciphertexts ct_1 and ct_2 , the decrypted product yields:

$$Dec(ct_1 \cdot ct_2) = m_1 \cdot m_2$$

III. MATRIX MULTIPLICATION BASED ON POLYNOMIAL PACKING

A. Motivation

GEMM constitutes a pivotal operation in a myriad of machine learning applications, contributing to approximately 70% of the total computational workload [19]. In the realm of FHE, data is encapsulated within polynomial structures to facilitate secure computation. Traditional batching techniques for matrix multiplication necessitate numerous rotational and multiplicative operations, leading to significant latency and the exacerbation of noise accumulation. Moreover, to uphold ciphertext security, polynomials must sustain a high degree, thereby imposing substantial spatial overhead.

To mitigate these challenges, we introduce an optimized polynomial-based data-packing methodology tailored for GEMM. This strategy is engineered to maximize the utilization of polynomial space, thereby reducing computational latency and minimizing noise accumulation while maintaining robust security guarantees.

B. Inner Product of Two Vectors

In homomorphic encryption, matrix multiplication efficiency can be significantly enhanced through strategic encoding of matrix entries as polynomial coefficients. Consider two one-dimensional matrices with k elements (where $2k-2 < n$):

$$A = [a_0, a_1, \dots, a_{k-1}], B = [b_0, b_1, \dots, b_{k-1}]^T$$

The result C of their matrix multiplication $A \cdot B$ can be computed as:

$$C = A \cdot B = \sum_{i=0}^{k-1} a_i b_i$$

To implement this polynomial packing method, we encode each element a_i and b_i as coefficients of polynomials. For matrix A , the elements are packed into the polynomial:

$$P_A(x) = a_0 + a_1x + a_2x^2 + \dots + a_{k-1}x^{k-1}$$

For matrix B , the elements are arranged in reverse order within the polynomial:

$$P_B(x) = b_{k-1} + b_{k-2}x + b_{k-3}x^2 + \dots + b_0x^{k-1}$$

And after polynomial multiplication, the result $P_C(x)$ can be expressed as:

$$\begin{aligned} P_C(x) = & a_0b_{k-1} + (a_0b_{k-2} + a_1b_{k-1})x \\ & + (a_0b_{k-3} + a_1b_{k-2} + a_2b_{k-1})x^2 \\ & + \dots \\ & + \left(\sum_{i=0}^{k-1} a_i b_i \right) x^{k-1} \\ & + \dots \\ & + (a_{k-1}b_0)x^{2k-2} \end{aligned}$$

Therefore, the result of the inner product of two arrays can be easily achieved in the coefficient of x^{k-1} .

C. Inner Product of A Matrix and A Vector

A straightforward way of packing for matrix multiplication is creating one polynomial for each row of matrix A and one polynomial for each row of matrix B . However, it is a waste of space and time to pack and encrypt each row individually.

One observation is that the inner product of two k -element arrays would use up $2k-1$ coefficients, where the coefficient in the middle is our result, and all the others contain some garbage that we do not

need. Therefore, when k is much smaller than n , we can use the other coefficients that are not polluted to store other valid results.

When consider the situation that two matrix that $A \in \mathbb{Z}^{r \times k}$ and $B \in \mathbb{Z}^{k \times 1}$.

$$A = \begin{bmatrix} a_{0,0} & a_{0,1} & a_{0,2} & \cdots & a_{0,k-1} \\ a_{1,0} & a_{1,1} & a_{1,2} & \cdots & a_{1,k-1} \\ a_{2,0} & a_{2,1} & a_{2,2} & \cdots & a_{2,k-1} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ a_{r-1,0} & a_{r-1,1} & a_{r-1,2} & \cdots & a_{r-1,k-1} \end{bmatrix}$$

$$B = [b_0 \ b_1 \ b_2 \ \cdots \ b_{k-1}]^T$$

We propose to pack the matrix in the following way:

$$P_A(x) = \underbrace{a_{0,0} + a_{0,1}x + \cdots + a_{0,k-1}x^{k-1}}_{A_0} + \underbrace{a_{1,0}x^k + a_{1,1}x^{k+1} + \cdots + a_{1,k-1}x^{2k-1}}_{A_1} + \cdots + \underbrace{a_{r-1,0}x^{(r-1)k} + \cdots + a_{r-1,k-1}x^{rk-1}}_{A_{r-1}}$$

$$P_B = b_{k-1} + b_{k-2}x + b_{k-3}x^2 + \cdots + b_0x^{k-1}$$

As shown in Section III-B, the inner product of A_0 and B will use the coefficients $[0, 2k-2]$, and the result is in the coefficient of $k-1$. Similarly, the inner product of A_1 and B will use the coefficients $[k, 3k-2]$, and the result is in the coefficient of $2k-1$. To better illustrate where the valid results are in each coefficient, we plot the results in Figure 1. We introduce ‘polluted space’ in the coefficient space of the resulting polynomial. Each result of matrix multiplication is generated at fixed slots $mk-1, m \in [1, r]$, while the regions between these slots represent polluted space and are irrelevant.

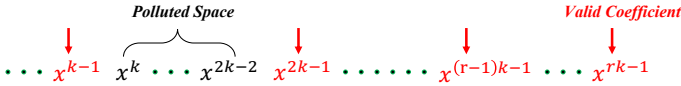


Fig. 1: Distribution of Valid Coefficients and Polluted Spaces in Polynomial Multiplication Results. Red dots indicate the positions of valid coefficients at multiples of k , while the black slots represent polluted spaces that require careful management in the polynomial encoding scheme.

The result of A_0B is valid only when the polluted space of $A_{r-1}B$ does not overlap with the valid result of A_0B , i.e., $rk-1+k-1 \leq n+k-2 \implies rk \leq n$. In other words, as long as the total number of elements in A is no larger than the polynomial degree n , the inner product of AB can be finished in one round, with all the results in the coefficients of x^{mk-1} , where $m \in [1, r]$.

D. Inner Product of Two Matrices

For the multiplication of two matrices, we can encode the matrices based on the size of matrix A to maximize the utilization of the coefficient space.

1) *When A is small ($rk \leq n$):* We could pack multiple columns of B into the same polynomial and finish in one shot. The number of columns that could be packed together is decided by $\lfloor n/(rk) \rfloor$. For example, consider two matrices A and B of sizes 2×3 and 3×2 respectively:

$$A = \begin{bmatrix} a_{00} & a_{01} & a_{02} \\ a_{10} & a_{11} & a_{12} \end{bmatrix}, \quad B = \begin{bmatrix} b_{00} & b_{01} \\ b_{10} & b_{11} \\ b_{20} & b_{21} \end{bmatrix}$$

And the result $C = A \times B$ is a 2×2 matrix.

$$C = \begin{bmatrix} c_{00} & c_{01} \\ c_{10} & c_{11} \end{bmatrix}$$

Assuming the polynomial degree $n \geq 2 \times 3 \times 2$, we can encode the matrices in the following way:

$$P_A(x) = (a_{00} + a_{01}x + a_{02}x^2) + (a_{10} + a_{11}x + a_{12}x^2)x^3$$

$$P_B(x) = (b_{20} + b_{10}x + b_{00}x^2) + (b_{21} + b_{11}x + b_{01}x^2)x^6$$

The result of polynomial multiplication is:

$$P_C(x) = P_A(x) \times P_B(x) = \cdots + c_{00}x^2 + \cdots + c_{10}x^5 + \cdots + c_{01}x^8 + \cdots + c_{11}x^{11} + \cdots$$

Compared to packing B with individual columns, this packing scheme can significantly improve the overall performance by reducing the number of packing and encryption.

2) *When A is large ($rk > n$):* Similar to the tiling technique in general matrix multiplication, we could split the row into multiple smaller tiles that can fit in one polynomial. For example, consider two matrices A and B of sizes 2×6 and 6×2 respectively:

$$A = \begin{bmatrix} a_{00} & a_{01} & a_{02} & a_{03} & a_{04} & a_{05} \\ a_{10} & a_{11} & a_{12} & a_{13} & a_{14} & a_{15} \end{bmatrix}, \quad B = \begin{bmatrix} b_{00} & b_{01} \\ b_{10} & b_{11} \\ b_{20} & b_{21} \\ b_{30} & b_{31} \\ b_{40} & b_{41} \\ b_{50} & b_{51} \end{bmatrix}$$

and the result $C = A \times B$ is a 2×2 matrix.

$$C = \begin{bmatrix} c_{00} & c_{01} \\ c_{10} & c_{11} \end{bmatrix}$$

For the ease of illustration, we assume the polynomial degree n is 3. We could encode matrix A using four polynomials, and could encode matrix B using four polynomials.

$$P_{A0}(x) = a_{00} + a_{01}x + a_{02}x^2, P_{A1}(x) = a_{03} + a_{04}x + a_{05}x^2$$

$$P_{A2}(x) = a_{10} + a_{11}x + a_{12}x^2, P_{A3}(x) = a_{13} + a_{14}x + a_{15}x^2$$

$$P_{B0}(x) = b_{20} + b_{10}x + b_{00}x^2, P_{B1}(x) = b_{50} + b_{40}x + b_{30}x^2$$

$$P_{B2}(x) = b_{21} + b_{11}x + b_{01}x^2, P_{B3}(x) = b_{51} + b_{41}x + b_{31}x^2$$

The result of polynomial multiplication is:

$$P_{C_{00}}(x) = P_{A0}(x) \times P_{B0}(x) + P_{A1}(x) \times P_{B1}(x)$$

$$P_{C_{01}}(x) = P_{A0}(x) \times P_{B2}(x) + P_{A1}(x) \times P_{B3}(x)$$

$$P_{C_{10}}(x) = P_{A1}(x) \times P_{B0}(x) + P_{A1}(x) \times P_{B1}(x)$$

$$P_{C_{11}}(x) = P_{A1}(x) \times P_{B2}(x) + P_{A1}(x) \times P_{B3}(x)$$

IV. HARDWARE ACCELERATION FOR THE BFV SCHEME

To evaluate our polynomial encoding method for GEMM, we develop hardware that implements the BFV homomorphic encryption. In our design, we take the Residue Number System (RNS) [20] and decompose the polynomial to q_{Num} distinct residue polynomials, each associated with a prime modulus q_i .

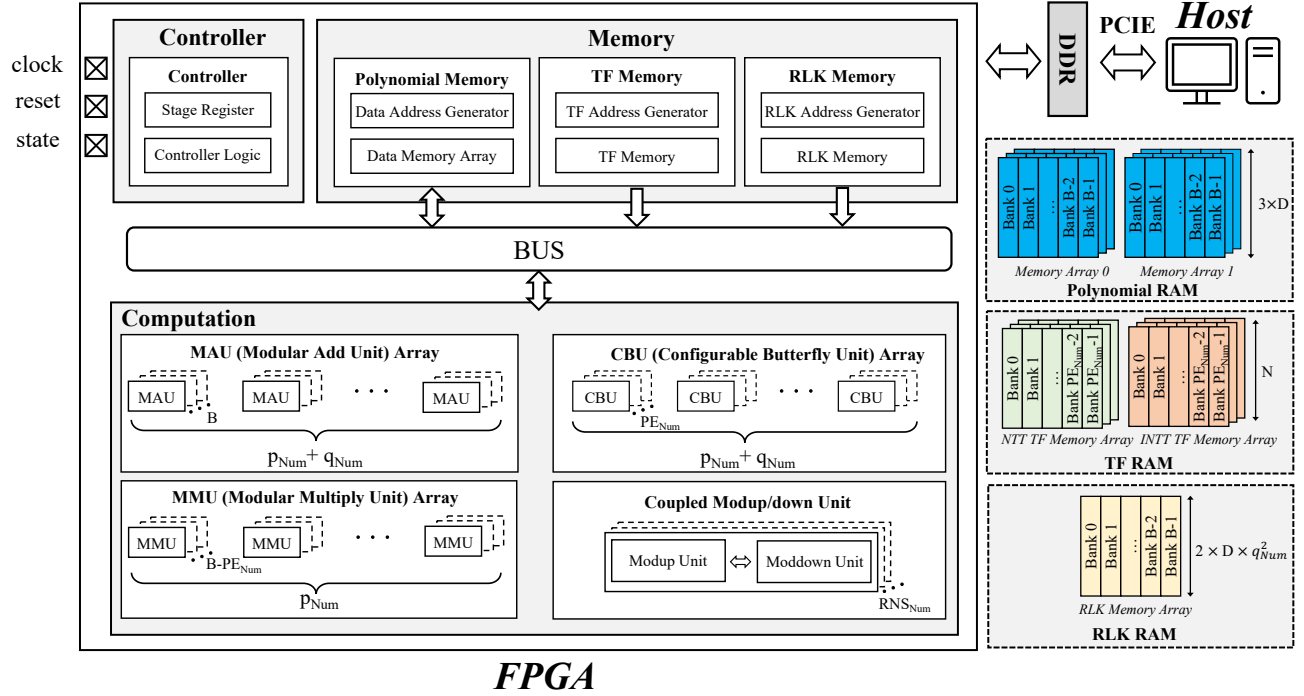


Fig. 2: Overall Architecture of Accelerator

A. Overall Architecture

The overall architecture, depicted in Figure 2, is organized into three primary components: the memory module, the controller module, and the computation module. Additionally, it features a DDR interface and connection to the host system through a PCIe interface.

The controller module functions as a state machine, generating control signals in response to specific stimuli. This module is essential for scheduling various operations, including Modup, Moddown, NTT, INTT, Relinearization, Keyswitch, and others. One key functionality of the controller module is to generate read/write (R/W) state signals to manage the data flow, involving Data Memory, Twiddle Factor (TF) Memory, and Relinearization Key (RLK) Memory. The control logic allows the configurable butterfly unit to form diverse data paths, enabling the execution of different computational tasks and control data flow among different modules.

The data memory module includes an address generation unit and $2 \times (q_{\text{Num}} + p_{\text{Num}})$ dual-port BRAM arrays, each comprising B memory banks ($B = \sqrt{N}$) with a depth of $3D$ for polynomial degree $N = DB$. The address generator produces R/W addresses based on control signals. The TF memory stores twiddle factors for all moduli q_i and p_j required for NTT and INTT operations, totaling $PE_{\text{Num}} \times (p_{\text{Num}} + q_{\text{Num}}) \times N$ factors. Each Ultra RAM in the TF memory contains PE_{Num} banks with a depth of N and a 72-bit width on the FPGA U250, storing NTT and INTT twiddle factors in the upper and lower 36 bits of each row, respectively. These factors remain constant after initialization and are accessed as directed by control signals. The RLK memory utilizes BRAM to store the RLK, which is essential for efficiently converting three polynomials into two during relinearization. The RLK memory array consists of B memory banks, each with a depth of $2D \times q_{\text{Num}}^2$. Due to hardware resource limitations, the KeySwitch operation stores the RLK in a single memory array instead of multiple arrays.

The computation module comprises the Configurable Butterfly Unit (CBU) module, Coupled ModUp/Down module, Modular Addition Unit (MAU) module, and Modular Multiplication Unit (MMU) module. The CBU module, designed based on the work in [21], consists of $(q_{\text{Num}} + p_{\text{Num}})$ CBU arrays, each containing PE_{Num} butterfly units. These configurable units perform operations such as Number Theoretic Transform (NTT), Inverse NTT (INTT), and modular multiplication, with each CBU array associated with a modulus q_i or p_j . The Coupled ModUp/Down module facilitates basis scaling and extension operations essential for ciphertext-ciphertext multiplication and comprises RNS_{Num} units. To address the computational intensity of the KeySwitch operation, we introduce additional MAU and MMU modules, enhancing performance through increased parallelism and efficiency. The MAU array consists of $(q_{\text{Num}} + p_{\text{Num}})$ arrays, each containing B units. The MMU array, which operates in conjunction with the CBU array and is primarily utilized during KeySwitch operations, includes P_{Num} arrays, each with $(B - PE_{\text{Num}})$ MMU units.

B. Polynomial Packing Inference Schedule

Figure 3 illustrates the inference pipeline leveraging polynomial packing optimization techniques. The proposed framework comprises six distinct computational stages: polynomial packing, homomorphic encryption, secure inner product computation, homomorphic decryption, coefficient extraction, and non-linear activation transformation. In our design, only the input data is encrypted to protect user privacy, while the model weights remain in plaintext form to optimize computational efficiency¹.

¹This design choice aligns with the typical privacy requirements in machine learning inference, where protecting user input data is paramount while model weights are considered public parameters.

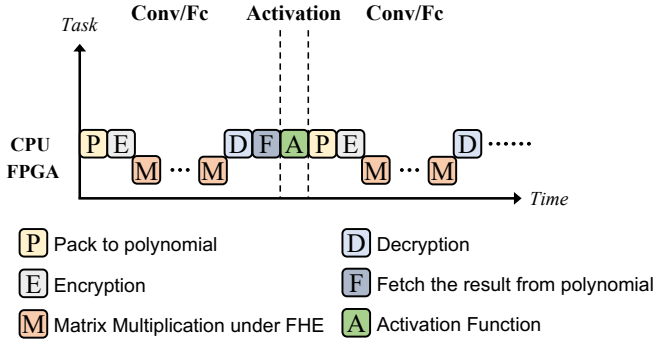


Fig. 3: Inference Schedule. The input data is encrypted to protect user privacy, while model weights remain in plaintext form for computational efficiency.

To achieve an optimal balance between computational efficiency and privacy preservation, we strategically partition the workload across heterogeneous computing platforms - executing the non-linear activation functions on plaintext data within the CPU environment while offloading the privacy-critical inner product operations (between encrypted inputs and plaintext weights) to the FPGA accelerator. This architectural design ensures both the confidentiality of sensitive user data and the computational efficiency of the inference pipeline.

V. EXPERIMENTAL EVALUATION

A. Experimental Setup

The proposed accelerator was implemented in Chisel, translated to Verilog, and then synthesized using the Xilinx Vivado toolchain version 2024.1. To evaluate the performance, we deployed the accelerator on the commercial FPGA platform, AMD Xilinx Alveo U250. The software reference implementation was developed using SEAL [22]. For the CPU host, we used a 12th Gen Intel® Core™ i7-12700 processor with 32GB of memory. The AMD Xilinx Alveo U250 features 1,728,000 Lookup Tables (LUTs), 3,456,000 Flip-Flops (FFs), 12,288 Digital Signal Processors (DSPs), 1,280 Ultra RAM, and 2,000 Block RAM. Additionally, the U250 is equipped with 16 GB of DDR memory and supports high-speed data transfer through a PCIe interface.

B. Benchmarks and Security Parameters

Benchmarks: For experimental evaluation, we adopted two representative CNN architectures from [5], which were originally designed for MNIST and CIFAR-10 classification tasks. To accommodate the integer-only arithmetic constraints of the BFV scheme, we performed uniform 8-bit quantization on both model weights and input features. We enhanced the network architecture by replacing the original *Square* activation with *ReLU* and incorporating *BatchNorm* layers after convolution operations, which improved the model's inference accuracy.

Security Parameters: The choice of BFV parameters significantly impacts ciphertext and plaintext data sizes, as well as computational complexity. To enable efficient inference without exceeding the plaintext modulus, we set the polynomial degree N to 2^{12} , the modulus q_i to 32 bits, and the total modulus Q to 224 bits.

C. Resource Utilization and MatMul Performance

Resource Utilization: In our hardware implementation, we configured the number of processing elements PE_{Num} to 8 and the number

of RNS bases RNS_{Num} to 4. After synthesizing and implementing our design on the Xilinx Alveo U250 FPGA platform, the resource utilization analysis shows that our design consumes **98.93%** of available Look-Up Tables (LUTs), **8.29%** of Flip-Flops (FFs), **28.8%** of Block RAM (BRAM), **4.375%** of Ultra RAM (URAM), and **26.75%** of Digital Signal Processors (DSPs).

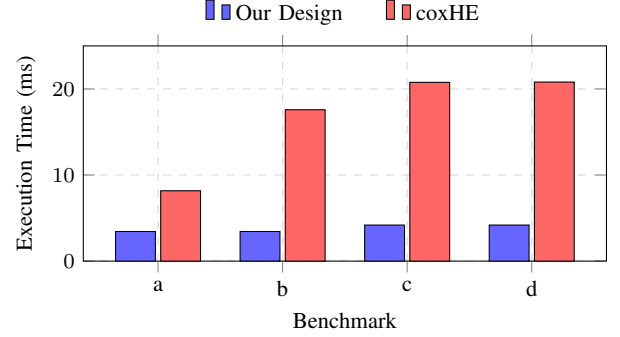


Fig. 4: Performance comparison between our design and the coxHE framework. For coxHE, benchmark **a** uses a polynomial degree of $N = 2^{12}$ with a 33-bit modulus (q_i). Benchmarks **b-d** employ $N = 2^{13}$ with 25-bit, 33-bit, and 43-bit modulus, respectively.

Small Matrix Multiplication: Matrix multiplication is the most time-intensive component of the inference process. Figure 4 presents a performance comparison between our design and the coxHE [23] framework, which is implemented under the CKKS scheme. Our evaluation focuses on the following matrix multiplication tasks: **a**: MatMul c-p: $(1 \times 2048) \cdot (2024 \times 1)$; **b**: MatMul c-p: $(1 \times 10) \cdot (10 \times 1)$; **c**: MatMul c-c: $(1 \times 2048) \cdot (2048 \times 1)$; **d**: MatMul c-c: $(1 \times 10) \cdot (10 \times 1)$.

In benchmarks **a** and **b**, our design achieves execution times of 3.439 ms, resulting in speedups of approximately $2.376\times$ and $5.114\times$ over coxHE's 8.17 ms and 17.58 ms, respectively. Similarly, in benchmarks **c** and **d**, our design reduces execution times to 4.188 ms, offering speedups of about $4.958\times$ and $4.967\times$ compared to coxHE's 20.766 ms and 20.800 ms, respectively. The superior performance of our design is attributed to the ability to pack all data into a single polynomial of degree 4096, enabling a one-time computation process. Additionally, the execution time differences between benchmarks **a** and **b** compared to benchmarks **c** and **d** arise from the varying number of clock cycles required for c-c and c-p operations, respectively.

Large Matrix Multiplication: For large-scale matrices, we implement a decomposition strategy that partitions the matrix into smaller polynomial segments. Each segment is encoded into a polynomial of length 4096, followed by the necessary multiplications. Subsequently, the results are extracted and aggregated to obtain the final product. Specifically, for matrix multiplication of size 8192×8192 , our hardware design completes the operation in 392,285.64 seconds under ciphertext operations. In stark contrast, our software implementation utilizing the SEAL library—employing a polynomial degree of 8192 and a coefficient modulus size of 218 bits—consumes approximately 7,413,087,362 seconds. This results in a speedup of approximately 1.89×10^4 , thereby demonstrating that our polynomial encoding method achieves substantial performance improvements for large-scale matrix multiplication.

D. Inference Performance on Benchmark Datasets

Accuracy: By optimizing the network architecture to incorporate more complex activation functions, we achieved superior accuracy compared to previous works. Specifically, on the MNIST dataset, our model attained an accuracy of **99.11%**, outperforming FxHENN [16] by 0.21% and Lola [24] by 0.16%. On the CIFAR-10 dataset, our model reached an accuracy of **78.99%**, surpassing both FxHENN and Lola by 4.89%.

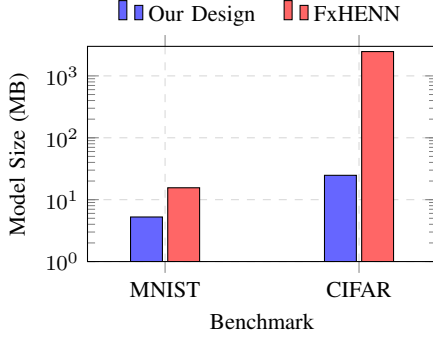


Fig. 5: Model Size of MNIST and CIFAR

Model Size: The model size is determined by the number of polynomials and the modulus size q . Our polynomial encoding method compresses the model by packing a significant number of weights into a single polynomial, thereby reducing the overall size. Figure 5 illustrates the model sizes for different benchmarks. For the MNIST dataset, our polynomial encoding occupies 15 MB compared to FxHENN's 15.57 MB, resulting in a 3.67% reduction in model size. Similarly, for the CIFAR dataset, our design requires 169.5 MB, whereas FxHENN demands 2,471.25 MB, reducing the model size by 93.14%. The model size is primarily influenced by the number of input and output channels, as the weights are packaged in a channel-first manner during the encoding process.

Performance: We compare the inference speed of our accelerator with prior works based on the BFV scheme, including CryptoNets [25], Lola [24], and Falcon [26], as well as with FxHENN [16] based on the CKKS scheme, using the MNIST and CIFAR datasets. Table I presents a detailed performance comparison.

TABLE I: Inference Performance Comparison

Implementation	Platform	MNIST (s)	CIFAR (s)
[25] [†]	Intel Xeon E5-1620L	205	-
[24] [†]	Azure B8ms (8 vCPUs)	2.2	730
[26] [†]	Azure B8ms (8 vCPUs)	1.2	107
[16] [†]	ALINX ACU15EG	0.19	54.1
Ours[†]	AMD Xilinx U250	0.045	13.53
Ours[§]	AMD Xilinx U250	0.077	27.3

[†] Encrypted inputs and plaintext weights.

[§] Encrypted inputs and encrypted weights.

As shown in Table I, our method significantly outperforms previous implementations by optimizing convolutional and fully connected layers. These optimizations reduce the inference latency to 0.045ms per image for encrypted inputs with plaintext weights and 0.077ms per image for fully encrypted inputs and weights. Compared to FxHENN, our accelerator achieves speedups of 4.22 $\times$ under encrypted inputs and plaintext weights and 2.47 $\times$ when both inputs and weights are fully encrypted. For inference on the CIFAR-10 dataset,

the experiment processes one image in just 13.53ms with plaintext weights and 27.3ms with encrypted weights, which are 3.99 $\times$ and 1.98 $\times$ faster than FxHENN.

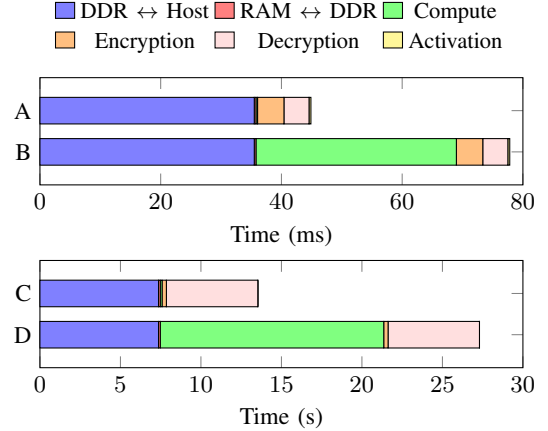


Fig. 6: Time Breakdown of Different Benchmarks. **A:** MNIST c-p; **B:** MNIST c-c; **C:** CIFAR c-p; **D:** CIFAR c-c

Time Breakdown: The time breakdown for the different benchmarks is presented in Figure 6. Benchmarks **A** and **C** involve encrypted inputs with plaintext weights, while benchmarks **B** and **D** involve both encrypted inputs and encrypted weights.

In scenarios with encrypted inputs and plaintext weights, data transformations between the host and the FPGA constitute the majority of the inference time, accounting for approximately **54%** to **79%**. Decryption and encryption processes contribute around **19%** and **44%** of the total time, depending on the benchmark. Conversely, in fully encrypted configurations, compute operations become the dominant factor, representing roughly **43%** to **51%** of the inference time. Additionally, data transformations still account for a significant portion, between **28%** and **46%**. These results highlight that data transmission remains a major contributor to inference latency across all benchmarks. However, in fully encrypted environments, the computational overhead introduced by encryption operations further elevates the overall latency.

VI. CONCLUSION

General Matrix-Matrix Multiplication with Fully Homomorphic Encryption presents significant challenges, including slow computation speeds, complex operations, and substantial overhead associated with processing encrypted data. To address these limitations, we introduce an enhanced data-packing method for matrix multiplication and extend this approach to accelerate both convolutional and fully connected layers. Additionally, we develop a novel computational schedule that incorporates more efficient activation functions, thereby improving network accuracy. To evaluate our method, we implemented dedicated hardware for homomorphic encryption using the BFV scheme. Deployed on the AMD Xilinx U250 FPGA platform, our hardware demonstrates exceptional performance in inference tasks on the MNIST and CIFAR-10 datasets. Compared to previous implementations, our hardware achieves significant speedups of up to 4.22 $\times$ for MNIST and 3.99 $\times$ for CIFAR-10, while also attaining higher accuracy. These results underscore the effectiveness of our enhancements in mitigating the inherent challenges of GEMM with FHE, leading to more efficient and accurate neural network inference.

REFERENCES

- [1] T. Zhu, D. Ye, W. Wang, W. Zhou, and P. S. Yu, "More than privacy: Applying differential privacy in key areas of artificial intelligence," *IEEE Transactions on Knowledge and Data Engineering*, vol. 34, no. 6, pp. 2824–2843, 2022.
- [2] B. Knott, S. Venkataraman, A. Hannun, S. Sengupta, M. Ibrahim, and L. van der Maaten, "Crypten: Secure multi-party computation meets machine learning," *Advances in Neural Information Processing Systems*, vol. 34, pp. 4961–4973, 2021.
- [3] Q. Yang, Y. Liu, T. Chen, and Y. Tong, "Federated machine learning: Concept and applications," *ACM Transactions on Intelligent Systems and Technology (TIST)*, vol. 10, no. 2, pp. 12:1–12:19, 2019.
- [4] F. Mo, A. S. Shamsabadi, K. Katevas, S. Demetriou, I. Leontiadis, A. Cavallaro, and H. Haddadi, "Darknetz: towards model privacy at the edge using trusted execution environments," in *Proceedings of the 18th International Conference on Mobile Systems, Applications, and Services*, 2020, pp. 161–174.
- [5] A. Brutzkus and A. Globerson, "Low-degree polynomial approximation of relu with applications to neural networks," in *International Conference on Machine Learning*. PMLR, 2019, pp. 902–910.
- [6] M. Naehrig, K. Lauter, and V. Vaikuntanathan, "Can homomorphic encryption be practical?" in *Proceedings of the 3rd ACM Workshop on Cloud Computing Security Workshop*. ACM, 2011, pp. 113–124.
- [7] I. Chillotti, N. Gama, M. Georgieva, and M. J. F. Izabachène, "Faster fully homomorphic encryption: Bootstrapping in less than 0.1 seconds," in *International Conference on the Theory and Application of Cryptology and Information Security*, ser. Lecture Notes in Computer Science, vol. 10031. Springer, 2016, pp. 3–33.
- [8] S. Halevi and V. Shoup, "Algorithms in helib," in *Annual Cryptology Conference*, ser. Lecture Notes in Computer Science, vol. 8616. Springer, 2014, pp. 554–571.
- [9] A. Costache and N. P. Smart, "Which ring-based somewhat homomorphic encryption scheme is best?" in *International Conference on Cryptography and Coding*, ser. Lecture Notes in Computer Science, vol. 10076. Springer, 2016, pp. 325–340.
- [10] R. Gilad-Bachrach, N. Dowlin, K. Laine, K. Lauter, M. Naehrig, and J. Wernsing, "Cryptonets: Applying neural networks to encrypted data with high throughput and accuracy," in *Proceedings of The 33rd International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, M. F. Balcan and K. Q. Weinberger, Eds., vol. 48. New York, New York, USA: PMLR, 20–22 Jun 2016, pp. 201–210. [Online]. Available: <https://proceedings.mlr.press/v48/gilad-bachrach16.html>
- [11] S. Fan, Z. Wang, W. Xu, R. Hou, D. Meng, and M. Zhang, "Tensorfhe: Achieving practical computation on encrypted data using gpgpu," in *2023 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, 2023, pp. 922–934.
- [12] X. Hu, Z. Li, Z. Wang, and X. Lu, "Alt: Area-efficient and low-latency fpga design for torus fully homomorphic encryption," *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, vol. 32, no. 4, pp. 645–657, 2024.
- [13] R. Agrawal, A. Chandrakasan, and A. Joshi, "Heap: A fully homomorphic encryption accelerator with parallelized bootstrapping," in *2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA)*, 2024, pp. 756–769.
- [14] Z. Xu, Y. Yang, R. Kannan, and V. K. Prasanna, "Bandwidth efficient homomorphic encrypted discrete fourier transform acceleration on fpga," in *2024 IEEE 32nd Annual International Symposium on Field-Programmable Custom Computing Machines (FCCM)*, 2024, pp. 1–12.
- [15] B. Reagen, W.-S. Choi, Y. Ko, V. T. Lee, H.-H. S. Lee, G.-Y. Wei, and D. Brooks, "Cheetah: Optimizing and accelerating homomorphic encryption for private inference," in *2021 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, 2021, pp. 26–39.
- [16] Y. Zhu, X. Wang, L. Ju, and S. Guo, "Fxnenn: Fpga-based acceleration framework for homomorphic encrypted cnn inference," in *2023 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, 2023, pp. 896–907.
- [17] Z. Brakerski, "Fully homomorphic encryption without modulus switching from classical gapsvp," in *Annual cryptology conference*. Springer, 2012, pp. 868–886.
- [18] J. Fan and F. Vercauteren, "Somewhat practical fully homomorphic encryption," *Cryptology ePrint Archive*, Paper 2012/144, 2012, <https://eprint.iacr.org/2012/144>. [Online]. Available: <https://eprint.iacr.org/2012/144>
- [19] E. Qin, A. Samajdar, H. Kwon, V. Nadella, S. Srinivasan, D. Das, B. Kaul, and T. Krishna, "Sigma: A sparse and irregular gemm accelerator with flexible interconnects for dnn training," in *2020 IEEE International Symposium on High Performance Computer Architecture (HPCA)*, 2020, pp. 58–70.
- [20] S. Halevi, Y. Polyakov, and V. Shoup, "An improved rns variant of the bfv homomorphic encryption scheme," in *Topics in Cryptology – CT-RSA 2019: The Cryptographers' Track at the RSA Conference 2019, San Francisco, CA, USA, March 4–8, 2019, Proceedings*. Berlin, Heidelberg: Springer-Verlag, 2019, p. 83–105. [Online]. Available: https://doi.org/10.1007/978-3-030-12612-4_5
- [21] X. Meng, Z. Jiang, and Y. Lyu, "Hf-ntt: Hazard-free dataflow accelerator for number theoretic transform," 2024. [Online]. Available: <https://arxiv.org/abs/2410.04805>
- [22] "Microsoft SEAL (release 4.1)," <https://github.com/Microsoft/SEAL>, Jan. 2023, microsoft Research, Redmond, WA.
- [23] M. Han, Y. Zhu, Q. Lou, Z. Zhou, S. Guo, and L. Ju, "coxhe: A software-hardware co-design framework for fpga acceleration of homomorphic computation," in *2022 Design, Automation & Test in Europe Conference & Exhibition (DATE)*. IEEE, 2022, pp. 1353–1358.
- [24] A. Brutzkus, R. Gilad-Bachrach, and O. Elisha, "Low latency privacy preserving inference," in *Proceedings of the 36th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, K. Chaudhuri and R. Salakhutdinov, Eds., vol. 97. PMLR, 09–15 Jun 2019, pp. 812–821. [Online]. Available: <https://proceedings.mlr.press/v97/brutzkus19a.html>
- [25] R. Gilad-Bachrach, N. Dowlin, K. Laine, K. Lauter, M. Naehrig, and J. Wernsing, "Cryptonets: Applying neural networks to encrypted data with high throughput and accuracy," in *International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 48. PMLR, 2016, pp. 201–210. [Online]. Available: <http://proceedings.mlr.press/v48/gilad-bachrach16.html>
- [26] T. Xu, M. Li, R. Wang, and R. Huang, "Falcon: Accelerating homomorphically encrypted convolutions for efficient private mobile network inference," 2023. [Online]. Available: <https://arxiv.org/abs/2308.13189>